
QwenPaw-Data: Bridging *Facts*, *Methodology*, and *Execution* for Autonomous Enterprise Data Analytics (Technical Report)

Tianjing Zeng[†], Yuntao Hong[†], Zhongjun Ding*, Dandan Liu*, Yanan Mei*,
Yunxiang Su*, Yiming Wang*, Xiaojian Zhang*, Jingyu Zhu*, Junhao Zhu*,
Zhuowen Liang, Jiazhen Peng, Lianggui Weng, Zhihao Ding, Kerui Yi, Qifeng Wang,
Rong Zhu[‡], Bolin Ding[‡], Liyu Mou[‡], Jingren Zhou[‡]

Alibaba Group

Abstract

Enterprise data analysis is emerging as a distinct frontier for autonomous agents. Compared with general-purpose interaction and software engineering, it operates in an open, ambiguous, and continuously evolving environment: business concepts must be grounded to the right entities, analytical procedures must be reproducible despite fuzzy feedback, and long-horizon workflows must execute over real enterprise data while preserving artifacts, provenance, and opportunities for human intervention. These characteristics call for a data-agent architecture that treats semantics, methodology, execution, and evolution as first-class system concerns. To this end, we introduce QwenPaw-Data, an agentic data system designed for enterprise intelligent data analysis. QwenPaw-Data consolidates heterogeneous assets from warehouses, dashboards, documents, interaction logs, and historical tasks into reusable, governable, and evolvable analysis assets, then turns natural-language requests into end-to-end analytical workflows spanning data understanding, retrieval, analysis, report generation, and decision support. Its architecture decomposes the problem into three collaborative subsystems: DataBridge provides trustworthy semantic grounding through interconnected metadata, knowledge, and trace graphs; Skill-Hub codifies expert analytical methodology into reusable and verifiable skills; and Host materializes these evidence and method assets into controllable, artifact-centric runtime execution. Across these subsystems, semantics, methods, traces, and feedback are continuously deposited back into the system, forming a self-evolving asset flywheel. Experiments on public benchmarks and real-world industrial BI workloads show that QwenPaw-Data improves both verifiable data-access capability and higher-level analytical quality, offering a practical foundation for reliable, traceable, and continuously improving enterprise data agents.

1 Introduction

1.1 Background

In recent time, the rapid evolution of Large Language Models (LLMs) has profoundly driven the development of autonomous *agents*, along with their underlying supporting frameworks, commonly referred to as *harnesses*. Both general-purpose agents (*e.g.*, chatting agents) and specialized agents (*e.g.*, coding) have achieved remarkable progress in autonomy, expanded task scope, and engineering practicality. Modern coding agents have transcended the early limitations of generating isolated

[†] Co-first authors.

^{*} Equal contribution (listed in alphabetical order).

[‡] Corresponding authors. Primary contact email: red.zr@alibaba-inc.com (Rong Zhu).

code snippets, acquiring the capability to independently manage task decomposition and debugging across the software development life cycle. Representative examples, such as OpenAI’s CodeX, Anthropic’s Claude Code, and frameworks like SWE-agent 2, demonstrate the ability to navigate complex codebases and resolve real-world issues. Currently, agent engineering has transitioned to scalable deployment, with industry surveys indicating that more than half of organizations have integrated agents into production environments [LangChain, 2026].

In the realm of data science, *data agents* have emerged as vital instruments for assisting data analysts (DA) and data scientists (DS). As an emerging “blue ocean” in the AI landscape, this field is increasingly attracting widespread attention for its capacity to drive profound business impact and strategic decision-making. Their primary technical directions focus on natural language interfaces (*e.g.*, Text-to-SQL), automated data exploration, multi-step reasoning, and in-depth insights generation. For instance, frameworks such as Din-SQL [Pourreza and Rafiei, 2023] and CHASE-SQL [Pourreza et al., 2025] leverage task decomposition and self-correction to optimize SQL generation, while end-to-end systems like Microsoft Fabric Data Agent [Microsoft, 2026a] and Snowflake Cortex Analyst [Snowflake, 2026] automate workflows from data ingestion to visual report generation. Researchers have substantially improved the accuracy of complex logical reasoning by incorporating lots of harness techniques, *e.g.*, Chain-of-Thought prompting and multi-agent collaboration. These breakthroughs lower technical barriers and improve the efficiency of extracting actionable insights.

Despite the rapid proliferation of general-purpose, coding, and data agents, some fundamental distinctions and core boundaries among these paradigms remain insufficiently delineated.

1. First, *what exactly are the fundamental distinctions and core boundaries that differentiate these paradigms from one another?*
2. Second, given these intrinsic characteristics and divergent operational environments, *is it genuinely necessary to establish a dedicated technological track exclusively for data agents, rather than merely adapting existing coding or general-purpose frameworks?*
3. Third, if such a distinct paradigm is indeed warranted, *what profound technical challenges must be overcome, and what specific enterprise demands should drive the tailored design of these specialized systems?*

In this technical report, we provide in-depth answers to these critical questions by systematically analyzing the unique characteristics and operational boundaries of enterprise data environments. Concurrently, we introduce our proposed solution, the QwenPaw-Data system, which is specifically architected to overcome these profound technical challenges and fulfill the tailored demands of real-world data analytics.

1.2 Data Agent: The Fundamental Difference

A natural question is why powerful general-purpose and coding agents, backed by ever-larger foundation models, do not already solve enterprise data analysis. As the claim summarized in [Anthropic, 2026], *data analysis is not software engineering*. We summarize the key differences in Table 1 and elaborate the details as follows.

General-purpose agents are primarily designed as universal frameworks to tackle a broad spectrum of everyday tasks, such as web navigation, operating system control, and general tool utilization. While they are capable of navigating dynamic, partially observable environments using multi-modal and heuristic feedback, they fundamentally lack the specialized mechanisms required for rigorous domain-specific problem-solving. Because they do not account for the stringent deterministic requirements of software engineering or the profound semantic ambiguity of enterprise data ecosystems, general-purpose agents are inherently ill-suited for complex coding and data analysis scenarios. Their generalized architectures fail to provide the domain-specific guardrails, structured verification loops, and deep contextual grounding necessary to ensure reliability and precision in these specialized fields.

Coding agents primarily tackle problems within a deterministic and closed verification environment. In software engineering, every step of the problem-solving process and the final output can be rigorously validated through a closed programming ecosystem, such as an Integrated Development Environment (IDE). Compilers, unit tests, and execution logs provide explicit, binary

Table 1: Comparative Analysis of General-Purpose, Coding, and Data Agents.

Dimension	General-Purpose	Coding	Data
Scenarios	Web/OS control, tool use	Code generation, debugging	Business analytics
Closedness	Dynamic, partially observable	Deterministic, closed	Open-space, ambiguous
Feedback	Multimodal, sparse, delayed	Explicit, binary, immediate	Fuzzy, subjective, human-reliant
Uniqueness	Diverse trajectories	Diverse implementations	Rigidly unique (single answer)
Search Cost	High (vast action space)	Light (flexible paths)	High (strict constraints)
Verification	Weak closed-loop	Hard closed-loop	No deterministic proof
Optimization	Universal framework; lacks specialized guardrails	Highly optimized for syntax, logic, and execution	Demands deep business context, semantic layers, and governance

feedback signals, enabling the agent to self-reflect, adjust, and iteratively refine its output in a “hard closed-loop”. Furthermore, the solution space for coding implementations is inherently diverse; as long as the functional objectives and test cases are satisfied, multiple valid algorithmic paths exist. This flexibility significantly alleviates the search burden on the model.

Conversely, the data analysis tasks addressed by data agents are inherently *open-space* problems. Each step in the analytical workflow involves open-ended exploration without deterministic mechanisms to prove correctness. Unlike code compilation, data queries lack explicit, automated feedback signals, rendering the system heavily reliant on external validation, particularly human feedback. Compounding this challenge is the profound ambiguity in both task descriptions and intermediate results—such as concept-to-entity mapping and business context interpretation—which makes the requisite feedback signals inherently fuzzy and subjective. Moreover, in contrast to the diverse solutions in coding, data analysis often demands a single, unique correct answer derived from a single authoritative source to inform a specific business decision. This rigid uniqueness, coupled with a singular decision logic and the absence of deterministic guardrails, drastically increases the search cost and complexity for the model, making autonomous analytics significantly more arduous than code generation.

In conclusion, traditional general-purpose and coding agents are fundamentally incapable of adapting to the intrinsic and stringent demands of data analysis scenarios. Therefore, the design and implementation of a dedicated data agent is not a mere elective, but an absolute imperative.

1.3 Agents for Enterprise Data Analysis: The Difficulties

Next, we analyze the challenges confronting data agents that primarily stem from two fundamental dimensions. The first arises from the intrinsic characteristics of data analysis tasks themselves, which involve open-ended exploration, ambiguous feedback, and rigid solution uniqueness, as discussed above. The second originates from the complex nature of real-world enterprise data ecosystems, which are inherently decentralized, multi-modal, and continuously evolving.

1.3.1 Difficulties from the Intrinsic Characteristics

To be more specific, the harness designed for data agent requires deep business context, semantic layers, and governance. Consider a seemingly simple data analysis request: “*analyze the average GAAP value of valid users for product X.*” We apply this simple question to explain the difficulties that a data agent need to resolve:

- **Grounding business concepts in the right data.** The request already hides two loaded terms, and most errors stem from failing to map such an ambiguous business concept onto the right data entity, which can break down in three compounding ways.
 - *Concept–entity ambiguity:* “valid users” can resolve to different underlying entities—the registered-user table, the login-session table, or the paid-subscription table—each a

legitimate mapping yet yielding a different number, while “GAAP value” must further be tied to the right revenue entity under the correct product scope.

- *Staleness*: schemas and metric definitions change constantly, so unmaintained knowledge silently returns subtly wrong results.
- *Retrieval failure*: even when the right entity exists and is well annotated, the vast search space keeps the agent from finding or using it correctly. The bottleneck is structured grounding, not access.
- **Codifying analytical methodology.** Even with the data grounded correctly, the average GAAP value is still only a headline number. Answering the question well calls for a reproducible procedure—segmenting the metric across dimensions, checking distributions, and attributing anomalies—a form of procedural knowledge that can neither be retrieved as a single “correct answer” nor reliably internalized in model parameters. Such methodology must also be pitched at the right level of abstraction: overly generic methods fail to capture business intent, while overly specific ones are bound to a single domain and lose transferability. Moreover, since data analysis lacks compiler-like deterministic verification, a standardized, reusable, and verifiable analytical methodology becomes the structural safeguard that replaces the missing verification.
- **Sustaining long-horizon, intrusive, artifact-centric execution.** Even with the right data and the right method, our example expands into a long chain of steps: querying the headline GAAP metric, detecting anomalies, decomposing and attributing it across dimensions, exploring new dimensions, and generating a report whose feedback is deposited for reuse. Such workflows are non-linear and long-running, directly execute SQL or Python over real business data, and produce reusable artifacts that must persist and be shared across sessions. This makes data analysis far more demanding than a short, self-contained action chain.

Unfortunately, as discussed in Section 1.2, existing general-purpose and coding agents do not fully address the above difficulties:

- General-purpose agents have no data-aware semantic model, so when pointed at a warehouse they must guess tables and fields (no grounding); they carry no codified analytical methodology (no methodology); and they are not designed for long-running, artifact-centered workflows (partial execution).
- Coding agents offer strong code generation and execution, so it is tempting to treat data analysis as a pure coding problem. Yet they too fall short on the same three dimensions: they can reach a schema but do not ground business semantics such as metric definitions (partial grounding); they encode no reusable analytical methodology (no methodology); and they run as a mostly linear action chain without long-horizon orchestration or cross-session artifact management (partial execution). As a result, a coding agent can produce runnable SQL yet still return a situationally wrong answer, and it cannot accumulate the analytical experience that future tasks depend on.

1.3.2 Difficulties from the Enterprise Data Ecosystems

Despite the advancements, the design of existing data agents predominantly leans toward idealized and academic scenarios, rendering them insufficient for addressing practical enterprise challenges. In real-world corporate environments, data ecosystems are characterized by three fundamental complexities as follows:

1. **Decentralization.** Enterprise data is highly fragmented and multi-modal, scattered across implicit human expertise, knowledge documents, relational databases, and unstructured formats, rather than residing in centralized, clean schemas.
2. **Ambiguity and Uncertainty.** Analytical requests from stakeholders are inherently vague, necessitating a multi-step verification and interactive clarification process to progressively align with the *exact* business intents.
3. **Growth and Evolution.** An effective analytical system must act as a growing entity that accumulates memory and self-evolves through fulfilled tasks, mirroring the cognitive development of human learning.

Consequently, current static and narrowly scoped data agent systems fail to meet the dynamic, interactive, and continuous learning requirements of authentic enterprise data environments. Recent evaluations utilizing benchmarks designed to simulate realistic enterprise scenarios corroborate this deficiency. For instance, assessments on benchmarks like DABench [Ma et al., 2026] and AgenticDataBench [Sun et al., 2026] reveal that even state-of-the-art LLM-based agents still struggle significantly with complex, real-world data analysis tasks, falling far short of the performance required for practical deployment.

1.4 QwenPaw-Data: Our Solution

To tackle the limitations of existing systems, we propose QwenPaw-Data, an agentic data system built specifically for enterprise data analysis scenarios. QwenPaw-Data consolidates the multi-source, multi-modal, and heterogeneous enterprise data, scattered across enterprise data warehouses, dashboards, business documents, interaction logs, and document knowledge bases, into reusable, governable, and evolvable analysis assets. Building on these assets, a user only needs to ask questions in natural language to interact with the agent to autonomously complete the end-to-end pipeline that spans data understanding, data retrieval, data analysis, report production, and ultimately decision support. At the same time, this system is strongly self-evolving, so every piece of user feedback and behavior helps it improve over time.

Unlike those systems, QwenPaw-Data is natively designed and implemented to resolve these challenges. It reaches this not through a stronger model, but with delicate harness techniques to decompose each of the three difficulties onto a component. In addition, we carefully define the interaction logic between them, enabling them to collaborate smoothly to tune every analysis into reusable assets. This gives QwenPaw-Data the following data-centric highlights that general-purpose and coding agents lack:

- **Trustworthy Semantic Grounding System.** QwenPaw-Data grounds every analysis in governed, graph-structured semantic evidence rather than prompt-level guessing, mapping ambiguous business concepts to the right data entities and eliminating data hallucination at its source.
- **Codified Analytical Methodology Hub.** QwenPaw-Data supplies reusable, verifiable analytical skills that encode how an expert analyzes, so answers follow a reproducible methodology instead of ad-hoc, one-off reasoning.
- **Controllable Long-horizon Execution Environment.** QwenPaw-Data orchestrates long-running, artifact-centric workflows with isolated SQL and Python execution, keeping every step traceable, verifiable, and open to mid-execution intervention rather than a restart.
- **Self-Evolving Data Assets.** Across all components in QwenPaw-Data, semantics, methods, and experience are deposited as governable, evolvable assets, so the system keeps improving the more it is used.

With QwenPaw-Data, enterprises can cover the serious analytical work a data team handles day to day, including but not limited to,

- **Business monitoring and anomaly diagnosis.** Track core metrics such as DAU, revenue, or conversion, and when one moves unexpectedly, automatically pinpoint which region, channel, or segment drove it.
- **Trend and growth analysis.** Analyze the access, retention, or conversion trend of a product, identify turning points, and attribute the changes behind them.
- **User and interaction insight.** Mine large volumes of dialogue and behavior logs to understand user intent, needs, and how they shift over time.
- **Periodic business reporting.** Produce end-to-end monthly or quarterly reports for a product or business line, from data retrieval to a finished, decision-ready document.
- **Ad-hoc deep-dive analysis.** Answer open questions such as “why did product X’s revenue drop last month?” with multi-dimensional decomposition and contribution attribution.

Across all of these, QwenPaw-Data does not stop at returning a number: it grounds every metric in the right business definition, follows an expert analytical methodology, and delivers traceable, decision-

Table 2: Comparison of representative data agent systems over four dimensions.

System	Semantic Grounding	Analytical Methodology	Execution Capabilities	Self-Evolution
Oracle Analytics AI Agents	✗	✗	✗	✗
Microsoft Fabric Data Agent	✓	✗	✗	✗
QuickSight / Cognos / Intelligent Q / BigQuery Analytics / Volcengine Agent	✗	✓	✓	✗
Databricks Genie Agent	✓	✓	✓	✗
QwenPaw-Data	✓	✓	✓	✓

Notation: ✓ explicit system-level support; ✗ partial, restricted, or insufficiently documented support; ✗ no explicit support or support limited to predefined functions.

ready results. By our evaluation (see Section 6), QwenPaw-Data substantially outperforms strong general-purpose agents on open-ended tasks and public benchmarks.

1.5 Comparison with Existing Systems

To position QwenPaw-Data within the current data-agent landscape, we also review and compare representative systems and commercial products across four key dimensions: semantic grounding, analytical methodology, execution capabilities, and self-evolution (which are discussed in Section 1.3.1 and Section 1.4). Table 2 presents a comparison over representative industrial systems across these four dimensions.

Oracle Analytics AI Agents [Oracle, 2026] and Microsoft Fabric Data Agent [Microsoft, 2026a] are comparatively assistant-oriented, but use different grounding mechanisms. Oracle configures agents through datasets, supplemental instructions, and knowledge documents, without exposing a standalone ontology or reusable skills. Microsoft Fabric Data Agent can use Fabric IQ Ontology [Microsoft, 2026b] for governed semantic grounding; however, its standalone runtime remains query-centric, selecting data sources and generating read-only SQL, DAX, or KQL, while broader workflows rely on external orchestration.

Amazon QuickSight [Amazon, 2026], IBM Cognos Analytics Assistant [IBM, 2026], Alibaba Quick BI Intelligent Q [Alibaba Cloud, 2026], Google BigQuery Conversational Analytics [Google, 2026], and Volcengine Intelligent Analytics Agent [Volcengine, 2026] are primarily BI-oriented agents, supporting multi-step analytical workflows for insight generation, dashboard construction, and report creation. However, their semantic grounding is generally provided through platform-specific resources without a specific semantic layer. Their analytical behavior is also primarily exposed through built-in BI functions or configured skills and workflows, lacking the right level of abstraction.

Databricks Genie Agents [Databricks, 2026a] is closest to QwenPaw-Data in overall direction, combining an automatically maintained Genie Ontology [Databricks, 2026b] with multi-step analytical execution and reusable skills. However, while its ontology evolves continuously, its analytical skills do not support persistent self-evolution.

In contrast, QwenPaw-Data provides explicit system-level support across all four dimensions, integrating governed semantic grounding, reusable analytical methodologies, controllable execution, and persistent self-evolution within a unified architecture.

2 QwenPaw-Data Architecture

2.1 Design Principle

The design principle of QwenPaw-Data is to decompose enterprise data analysis according to the core questions an AI-native data agent must answer: what evidence to trust, what analytical method to follow, and how to execute the resulting workflow in a controllable way. A loose analogy can be drawn to how enterprise analysis teams collaborate across data modeling, BI methodology, and

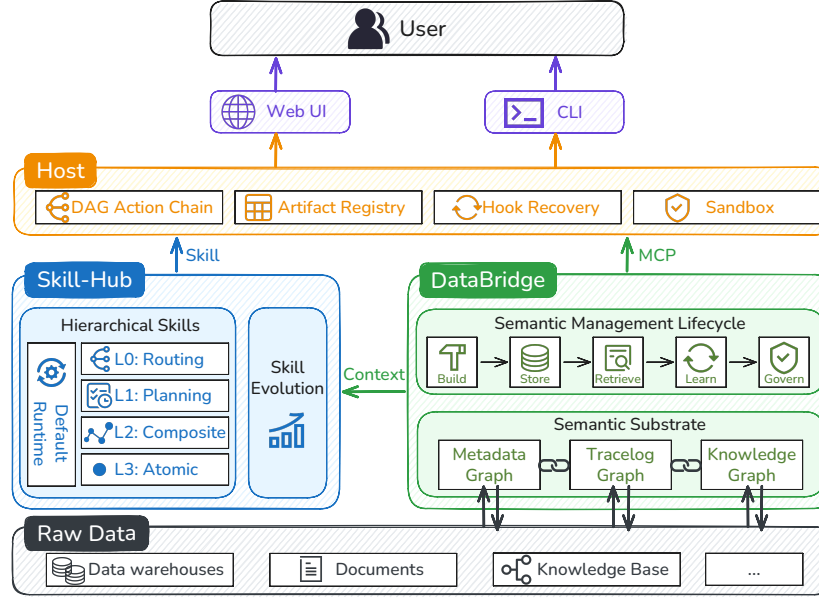


Figure 1: The architecture overview of QwenPaw-Data.

execution infrastructure. The analogy serves as an intuitive aid; the architecture itself is defined by agent-system responsibilities.

This decomposition directly responds to the three difficulties identified in Section 1.3.1: grounding business concepts in the right data calls for an evidence-grounding subsystem; codifying analytical methodology calls for a method-orchestration subsystem; and sustaining long-horizon, intrusive execution calls for an execution-control subsystem. Accordingly, QwenPaw-Data decomposes an intelligent data analysis system into three decoupled modules with orthogonal responsibilities:

- **DataBridge** — evidence grounding. It answers *what facts to use*: it grounds business semantics, tracks staleness, and retrieves the minimum sufficient evidence for each task. Intuitively, it plays a role close to the semantic and modeling support provided by data warehouse teams.
- **Skill-Hub** — method orchestration. It answers *how to analyze*: it standardizes reusable analytical methods, workflows, procedural conventions, and quality expectations. Intuitively, it resembles the methodological and reporting expertise accumulated by BI teams.
- **Host** — execution control. It answers *how to run*: it materializes workflows as DAGs, isolates SQL and Python execution in sandboxes, invokes tools and subagents, and manages artifacts, traces, and recovery. Intuitively, it reflects the execution and infrastructure support usually provided by data/platform engineering teams.

The reason for keeping the three modules decoupled is that they evolve at different rates and along different axes. Semantic evidence changes frequently as the business evolves; analytical methods change slowly and are reusable across domains; runtime infrastructure is largely stable. By separating them, each module can be developed, debugged, and upgraded independently, and upgrading any one module does not require restarting the other two. By our evaluation in Section 6.2, each component contributes to improve the system performance.

2.2 System Overview

Figure 1 illustrates the overall architecture of QwenPaw-Data. At the bottom, enterprise data assets are heterogeneous by nature: analytical tables, documents, searchable knowledge, historical traces, and operational data are scattered across different storage systems. DataBridge turns these scattered sources into a governed semantic substrate. Concretely, it organizes three interconnected graphs:

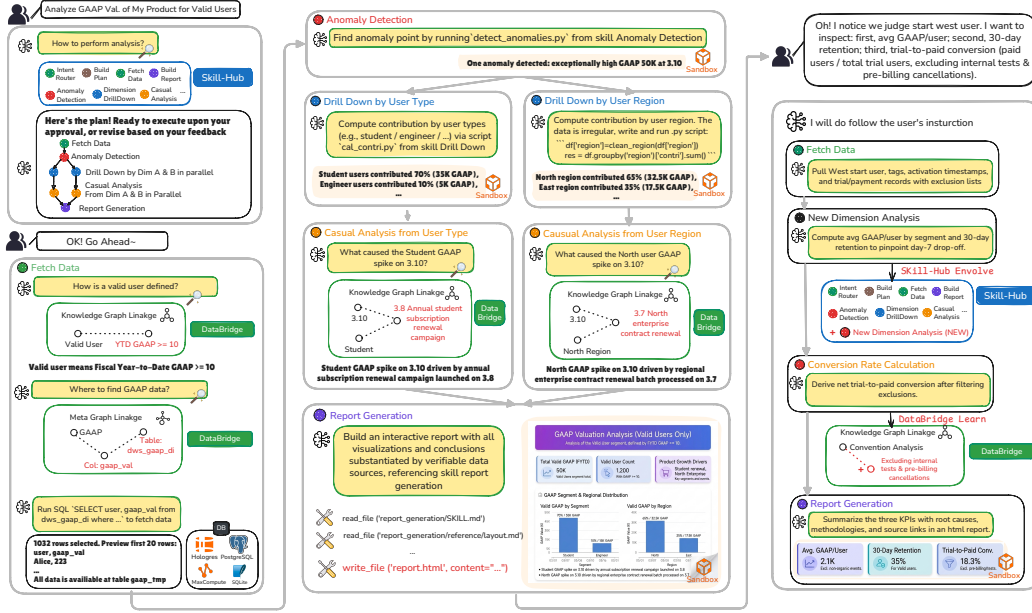


Figure 2: Illustration of how QwenPaw-Data empowers enterprise operations analytics.

the *Metadata Graph (MG)* describes databases, tables, columns, metrics, dimensions, and lineage; the *Knowledge Graph (KG)* captures business entities, definitions, rules, and organizational context; and the *Trace Graph (TG)* records task traces, tool usage, intermediate artifacts, user feedback, and reusable experience. Downstream components do not retrieve raw fragments directly; they request task-specific semantic evidence from DataBridge.

Skill-Hub sits above this semantic substrate as the method subsystem. It is organized as a hierarchy of reusable skills: routing skills define how task families are identified, planning skills provide templates for decomposing user intent into executable analytical plans, workflow skills describe complete workflows such as diagnosis or report construction, and atomic skills package fine-grained analytical operations such as anomaly detection, dimensional drill-down, attribution, visualization, and evidence summarization. Importantly, Skill-Hub does not execute these skills by itself; it provides skill specifications, references, and scripts that Host can interpret and run. The same skills consume semantic evidence from DataBridge rather than encoding changing business facts inside the skill itself, which keeps analytical methods reusable across domains.

Host is the runtime subsystem that makes the semantic and methodological subsystems executable. It reads the skill specifications, references, and scripts provided by Skill-Hub, turns them into a DAG execution graph, schedules independent branches in parallel, exposes tools for SQL, Python, file operations, and report construction, runs tool calls inside isolated sandboxes, and maintains an artifact registry that links each intermediate result back to its producing action. The three modules communicate through stable contracts: Skill-Hub supplies the analytical method assets, DataBridge supplies the governed evidence needed at each step, and Host executes the plan while preserving state, artifacts, and recovery points.

2.3 An End-to-End Walkthrough

Figure 2 traces a single request—"analyze the average GAAP value of valid users for product X"—through the system, showing how DataBridge, Skill-Hub, and Host collaborate during a real analytical task.

Plan. The user first asks the question in natural language. Host consults Skill-Hub to select the relevant routing and planning skills, then uses their specifications and references to decompose the request into a DAG-shaped workflow: fetch data, detect anomalies, drill down by user type and region in parallel, conduct causal analysis, and generate the final report. Host materializes this plan as an

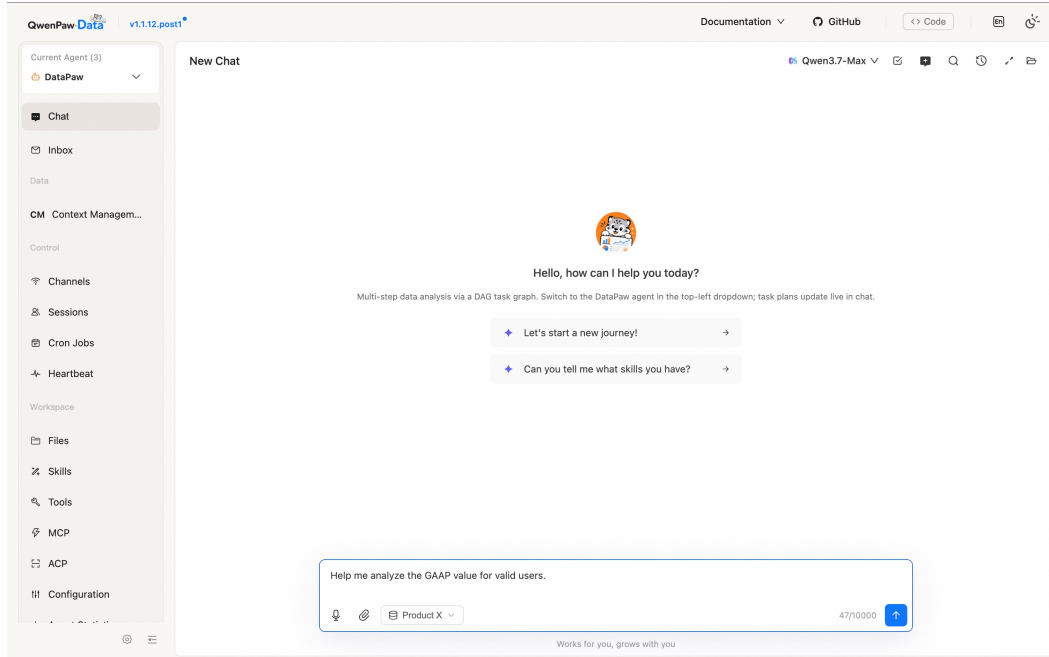


Figure 3: The ChatWeb interface of QwenPaw-Data, delivered as a plugin on top of QwenPaw.

executable DAG, keeps the plan inspectable, and waits for the user’s approval or revision before running it. DataBridge can already participate at this stage by supplying high-level semantic hints that constrain what metrics, entities, and dimensions the plan should consider.

Data retrieval. Once execution starts, the first question is not how to write SQL, but what the business terms mean. DataBridge resolves “valid users” through the Knowledge Graph and locates the GAAP metric through the Metadata Graph, linking the logical definition to the physical table and column. Host then invokes the corresponding data-access tools and executes the generated SQL in a sandboxed environment against the underlying warehouse. The resulting dataset is registered as an artifact, so later conclusions can be traced back to the exact query, source, and previewed rows.

Analysis. During analysis, Host follows the reusable skill assets from Skill-Hub, including scripts and references for anomaly detection, dimensional drill-down, contribution calculation, and causal analysis. Host schedules independent branches of the DAG in parallel—for example, user-type and region drill-downs can run concurrently—and runs each script or tool call inside an isolated sandbox. During attribution, DataBridge continues to provide grounded evidence from KG and TG, linking observed metric changes to business events, historical traces, or known rules rather than letting the model guess from prompt context alone.

Report generation. After the analytical branches converge, Host uses the report-generation skill package from Skill-Hub to organize findings, charts, methods, and source links into a decision-ready report. Host provides the file and rendering tools, manages generated figures and intermediate tables, and records the report in the artifact registry. DataBridge supplies provenance and semantic references, so each headline conclusion in the report remains connected to the metric definition, retrieved data, and analysis path that produced it.

Self-evolution. The task does not end when the report is delivered. The execution trace, user feedback, new analysis demand, and confirmed metric definitions become update signals for the two asset layers. Skill-Hub can turn a newly requested procedure, such as a new-dimension analysis, into a candidate reusable skill; DataBridge can promote validated semantic corrections into MG, KG, or TG; and Host provides the traces, artifacts, and execution state needed to make this evolution auditable. In this way, a completed task becomes reusable experience for the next similar analysis.

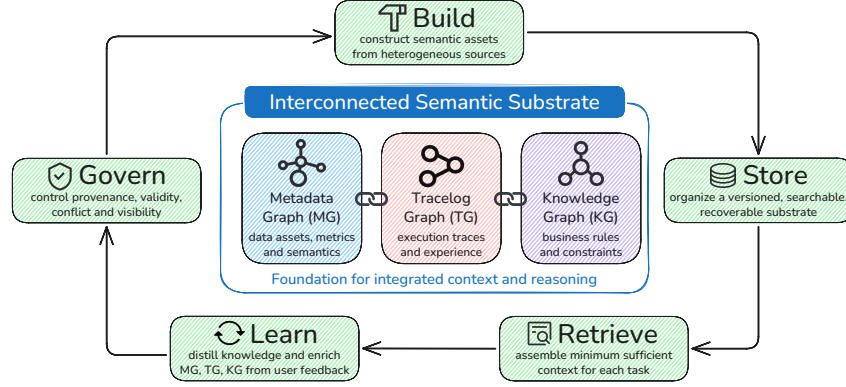


Figure 4: The overall structure of DataBridge. The center shows the semantic evidence stores, while the outer loop shows how evidence is built, stored, retrieved, learned, and governed.

2.4 How to Use QwenPaw-Data

QwenPaw-Data provides two major usage modes: ChatWeb and CLI/SDK. ChatWeb targets interactive business analysis, while CLI/SDK targets platform integration and secondary development.

ChatWeb. In ChatWeb mode, QwenPaw-Data is delivered as a plugin on top of QwenPaw [Alibaba Group, 2026], a general-purpose agent platform, and reuses its conversational frontend, so an existing chat interface is extended with data-analysis capabilities without rebuilding the interaction layer. The frontend is further adapted for data workloads, for example to visualize the parallel DAG task plan and to track long-running, multi-step analyses, as shown in Figure 3. Through ChatWeb, users submit analytical requests in natural language, inspect the generated plan, track task progress, view intermediate artifacts, and continue refining the analysis in the same conversation. This mode suits business users and analysts who need an interactive data-analysis experience.

CLI/SDK. CLI/SDK mode is designed for secondary development and platform integration. QwenPaw-Data exposes its core capabilities, including intent understanding, task planning, and workflow execution, through a lightweight command-line interface, so it can be embedded into existing enterprise BI platforms without a dedicated web frontend. Developers can extend data sources, register new tools, customize analytical skills, and define domain-specific semantic rules. As a result, QwenPaw-Data can be seamlessly embedded into existing enterprise data platforms, internal BI systems, workflow engines, or other service environments, serving as an infrastructure layer for building customized data agents.

3 DataBridge

3.1 Role: The Semantic Evidence Subsystem

DataBridge is the semantic evidence subsystem of QwenPaw-Data. Its responsibility is to turn fragmented enterprise data, business knowledge, and historical task experience into governed evidence that Host can retrieve and use during analytical execution. This is the component that supports trustworthy semantic grounding: instead of asking the model to infer from prompt-level context, DataBridge exposes verified, traceable, and governable evidence for each task.

This role is necessary because enterprise analysis is an open-form task. Business concepts are ambiguous, metric definitions change over time, and useful experience is scattered across past tasks, documents, data models, and user feedback. DataBridge therefore does not behave like a generic text retriever. It maintains a semantic evidence substrate that can be built from heterogeneous sources, stored with provenance and validity, retrieved as task-specific context, improved from feedback, and governed when definitions become stale or conflicting.

Figure 4 provides the high-level map of DataBridge. The center represents the semantic substrate that stores different kinds of evidence, while the surrounding lifecycle explains how evidence enters the

system, becomes retrievable, returns to Host during task execution, and evolves through feedback and governance. The rest of this section follows this logic: we first introduce what kinds of information DataBridge manages, then explain how the lifecycle operates over them.

3.2 What DataBridge Manages

Before introducing the storage structure, it is useful to clarify what kinds of information DataBridge manages. In enterprise analysis, the evidence needed by an agent is not a single document or table schema, but a combination of three information types.

Data semantics. Data semantics describes the structure and meaning of data assets: databases, tables, columns, metrics, dimensions, lineage, partitions, and data-source relationships. In the GAAP use case of Figure 2, this information tells Host where the GAAP metric is physically stored, how it maps to the underlying table and column, which product and time fields must be used, and which analytical dimensions such as user type or region are available for drill-down.

Business knowledge. Business knowledge captures concepts and rules that are not fully encoded in schemas. It includes business entities, metric definitions, operational rules, policies, events, and external definitions. In the same use case, this is where DataBridge represents the definition of “valid users,” GAAP-related business rules, and business events such as subscription-renewal campaigns that may explain metric changes.

Task experience. Task experience records what has happened in previous analytical tasks: execution traces, tool calls, intermediate artifacts, successful analysis paths, failed attempts, user corrections, and reusable lessons. For recurring GAAP analyses, this information helps the system remember which drill-down branches were useful, which scripts or tool calls produced reliable evidence, which assumptions users corrected, and which historical traces should be reused during attribution.

3.3 Semantic Evidence Lifecycle

DataBridge manages the three information types through a five-stage lifecycle: Build, Store, Retrieve, Learn, and Govern. The lifecycle is designed to separate evidence construction from evidence use: information is first collected and normalized, then instantiated as governed graph assets, retrieved as task-specific context, improved from usage, and continuously controlled before it can influence future analytical execution. Figure 5 illustrates this lifecycle with the GAAP analysis example, which we follow throughout this subsection.

3.3.1 Build: Construct Candidate Evidence

The Build stage converts heterogeneous enterprise sources into candidate semantic evidence. Its input includes structured metadata from warehouses and metric systems, semi-structured knowledge from documents and dashboards, and behavioral signals from historical tasks, reports, intermediate artifacts, user feedback, and failure records. These sources are collected together because enterprise analysis usually depends on evidence scattered across systems rather than on a single schema or document.

The main function of Build is normalization. DataBridge extracts candidate entities, relations, definitions, and traces from raw sources, aligns them to the three information types, and attaches source information and preliminary confidence signals. At this stage, the output is not yet trusted graph knowledge; it is candidate evidence prepared for graph storage and later governance. In the GAAP use case, Build may extract a candidate GAAP metric from warehouse schemas, capture the business concept of “valid user” from business reports and knowledge bases, and mine reusable experience from chat history, such as the observation that left joining `dws_gaap_di` with `dim_user` is efficient. Together, these candidates later help the LLM understand what “GAAP for valid users” actually refers to, where the underlying data lives, and how to retrieve it efficiently, so that similar analyses can reuse this evidence instead of rediscovering everything from scratch.

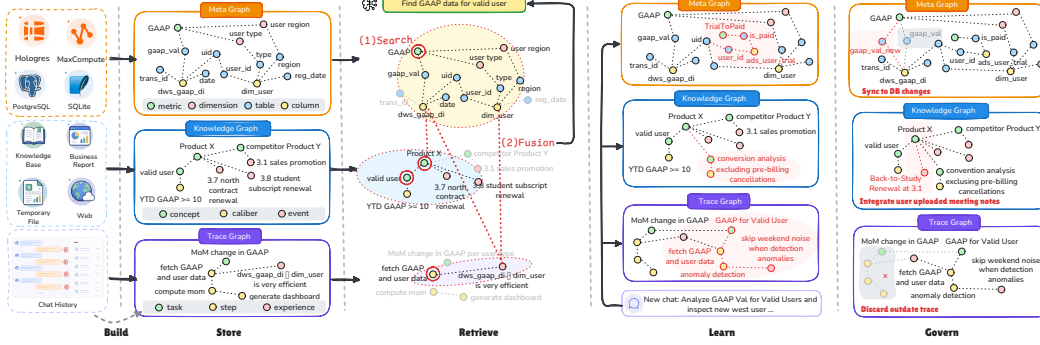


Figure 5: The semantic evidence lifecycle of DataBridge on the GAAP use case, spanning five stages: Build ingests and normalizes heterogeneous sources, Store instantiates them as three semantically interconnected graphs, Retrieve assembles task-specific evidence via cross-graph search and fusion, Learn distills new metrics, rules, and experiences from usage, and Govern keeps the evidence reliable.

3.3.2 Store: Instantiate Evidence as Graph Assets

The Store stage turns candidate evidence into persistent graph assets. It does not simply archive the three information types; it instantiates them as typed nodes, typed edges, and cross-graph links that can be retrieved and governed:

- *Metadata Graph (MG)* represents data assets as schema-level nodes, such as databases, tables, columns, metrics, dimensions, and partitions. Edges encode structural and semantic relations, such as table–column containment, metric–column mapping, metric–dimension compatibility, and lineage between upstream and downstream assets.
- *Knowledge Graph (KG)* represents business knowledge as concept-level nodes, such as business entities, metric definitions, rules, policies, events, and external concepts. Edges encode relations such as definition, constraint, scope, temporal applicability, and links from business concepts back to the MG assets that implement them.
- *Trace Graph (TG)* represents task experience as execution-level nodes, such as tasks, plans, tool calls, intermediate artifacts, reports, feedback, and corrections. Edges encode how an analysis was executed, including which action produced which artifact, which evidence was used, which branch succeeded or failed, and which user correction updated MG or KG evidence.

Each node and edge is stored with governance metadata, including provenance, trust level, temporal validity, lifecycle state, and visibility. This metadata is part of the graph representation itself, so downstream retrieval can distinguish verified, outdated, conflicting, or restricted evidence. In the GAAP use case, Store turns these candidates into typed nodes and edges, each supporting a later step. In MG, the `dws_gaap_di` table contains the `gaap_val` column that the GAAP metric maps to, which grounds data retrieval, and the metric stays compatible with the user type and region dimensions used for drill-down analysis. In KG, the “valid user” calibration $YTD\ GAAP \geq 10$ fixes the filter applied when fetching data, while renewal events such as the student-subscription campaigns feed the later causal analysis. In TG, the prior task, its steps, and its artifacts, together with the reusable experience distilled by the Learn stage (Section 3.3.4), speed up a follow-up similar request with the efficient join it already found.

3.3.3 Retrieve: Assemble Task-Specific Evidence

The Retrieve stage assembles the minimum sufficient evidence needed by a concrete task. Given the user request, current plan, and session state from Host, DataBridge anchors task mentions to graph nodes instead of retrieving raw text chunks. It then expands task-specific subgraphs across MG, KG, and TG, fuses the selected evidence, and returns a structured context package or concise natural-language answer back to Host.

The design goal of Retrieve is precision under governance. MG contributes concrete data assets and valid analytical dimensions; KG contributes business definitions, rules, scopes, and events; TG contributes prior execution experience, reusable paths, and known pitfalls. Retrieve combines these signals while respecting provenance, validity, visibility, and conflict status, so Host receives evidence that is compact enough to act on and reliable enough to ground execution. In the GAAP use case, for a request such as “find GAAP data for valid users,” Retrieve anchors three mentions to graph nodes: the GAAP metric in MG, the “valid user” concept together with the current product in KG, and the prior fetch-data step in TG. Each anchor then expands outward into the semantically related context around it, and the resulting subgraphs are fused across layers, for instance by linking the `dws_gaap_di` and `dim_user` tables in MG to the join experience carried by that fetch-data step in TG. Together they assemble a compact yet complete context, so Host works from grounded, traceable evidence.

3.3.4 Learn: Convert Usage into Reusable Evidence

The Learn stage turns real analytical usage into reusable evidence. It has two complementary paths: online learning absorbs explicit feedback during task execution, such as user-confirmed definitions, corrected rules, rejected assumptions, and report review comments; offline learning mines accumulated traces for recurring patterns, stable metric–dimension relations, useful drill-down paths, and verified failure modes.

The output of Learn is a set of candidate updates to MG, KG, or TG. These updates are intentionally separated from direct execution. A correction may suggest a KG rule update, a repeated successful workflow may suggest a TG experience pattern, and a verified metric–column relation may strengthen an MG mapping. This preserves the boundary between DataBridge and Skill-Hub: DataBridge evolves semantic evidence, traces, and rules, while Skill-Hub evolves reusable method assets such as skill packages, references, and scripts. In the GAAP use case (Figure 2), Learn works on the execution trace and explicit user feedback accumulated across the whole analysis, distilling three candidate updates. The user’s definition of net conversion becomes a new trial-to-paid conversion metric in MG, along with its supporting columns and table. A correction the user made turns into a KG rule *excluding pre-billing cancellations*. The successful run, meanwhile, leaves TG a reusable experience, *skipping weekend noise during anomaly detection*. Once promoted, DataBridge supplies these updates as evidence for later requests, so the agent automatically conducts subsequent analyses under the same definition, exclusion, and caveat; this reduces the attention the user must devote to restating them and improves the efficiency of recurring analyses. Until promotion, they remain candidates rather than one-off session notes.

3.3.5 Govern: Control Evidence Quality and Access

The Govern stage keeps the evidence layer reliable over time. It controls whether graph evidence is allowed to affect retrieval and execution. DataBridge checks provenance, temporal validity, conflicts, trust level, lifecycle state, and visibility; promotes validated candidate updates; deprecates stale nodes; and prevents conflicting or unauthorized evidence from being used silently.

Govern is both an automated control process and a human-facing correction process. DataBridge exposes visual governance interfaces for MG, KG, and TG, allowing users to inspect the three graphs and directly create, update, or delete graph nodes, edges, rules, and trace records when human correction is required. These operations update the governed evidence layer rather than patching a single prompt or session. In the GAAP use case, Govern updates, adds, and removes evidence to keep the three graphs aligned with reality. When an upstream schema change in the `dws_gaap_di` table renames the GAAP column from `gaap_val` to `gaap_val_new`, it updates MG by deprecating the old field and redirecting to the new one, so that retrieval never binds to a dropped column. Also, in response to user-uploaded meeting notes that surface a previously unrecorded campaign, it adds a new *Back-to-Study Renewal* event to KG on March 1. And as time passes, a trace or experience in TG that is no longer used or whose definition no longer holds is removed before it can mislead future attribution. Through these operations, only current and trustworthy evidence reaches Retrieve, and superseded or conflicting evidence never silently influences execution.

3.4 What DataBridge Guarantees

Through the interaction of the three information types and the five-stage lifecycle, DataBridge provides three guarantees for QwenPaw-Data:

- *Groundedness*: business concepts are mapped to verified graph evidence rather than inferred from prompt context.
- *Freshness*: definitions, schemas, and business rules carry validity and lifecycle state, so stale evidence can be detected and governed.
- *Evolvability*: user feedback, execution traces, and confirmed corrections become reusable semantic assets for future tasks.

Together, these guarantees allow Host to execute analytical workflows over governed evidence rather than over fragmented, unverified context.

4 Skill-Hub

4.1 Role: The Method Asset Subsystem

Skill-Hub is the method asset subsystem of QwenPaw-Data. Its responsibility is to turn reusable analytical methodology into structured skill assets that Host can interpret and execute during data-analysis tasks. If DataBridge answers *what facts to use* and Host answers *how to run*, Skill-Hub answers *how to analyze*: which analytical path to follow, which operations are required, what intermediate artifacts should be produced, and how the final result should be checked and organized.

This role is necessary because enterprise data analysis lacks the hard closed-loop verification available in software engineering. A compiler or unit test can tell a coding agent whether a program is syntactically valid or functionally correct, but a data-analysis task usually depends on business definitions, analytical assumptions, and human judgment. Skill-Hub therefore provides a soft verification loop: it codifies expert analytical procedures into reusable assets with explicit inputs, outputs, expected artifacts, and evaluation criteria, so Host does not rely only on ad-hoc model reasoning.

Skill-Hub is not an execution runtime. It provides skill specifications, references, scripts, and skill packages; Host reads these assets, materializes executable plans, invokes tools, runs scripts, manages sandboxes, and records artifacts. This separation keeps analytical methods reusable across data domains while allowing DataBridge and Host to evolve independently.

Figure 6 summarizes the two core ideas of Skill-Hub: hierarchical organization and continuous evolution. The hierarchy decomposes analytical methodology from task routing down to atomic analytical operations, while the evolution loop keeps these method assets updated through evaluation, historical traces, and user feedback.

4.2 What Skill-Hub Manages

Before introducing the hierarchy, it is useful to clarify what kinds of assets Skill-Hub manages. A skill is not just a prompt or a script. Following the commonly adopted skill organization, each skill package is mainly composed of a *skill specification*, *references*, and *scripts*. Together, they form a reusable method package that can be interpreted by Host and grounded with evidence from DataBridge.

Skill specifications. Skill specifications, typically written in SKILL.md, define the structured contract and procedural logic of an analytical method. They describe the task types a skill applies to, the semantic evidence it expects from DataBridge, the inputs Host should provide, and the outputs and artifacts that should be produced. More importantly, this is where the analysis workflow is specified: the procedure steps, step dependencies, branching conditions, revision points, and quality expectations are all described at the method level. Host then interprets this specification, materializes the workflow as an executable DAG, and manages the actual execution.

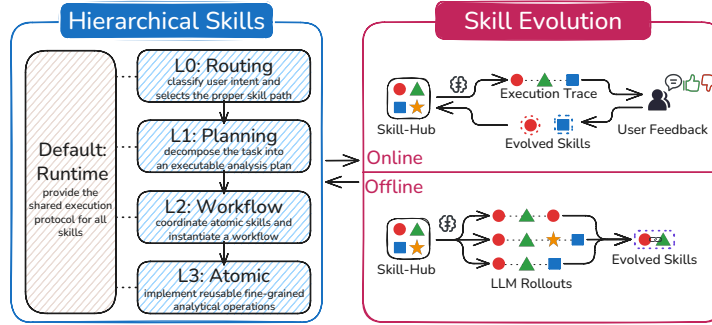


Figure 6: Hierarchical skills and skill evolution in Skill-Hub. The left part shows the L0–L3 skill hierarchy (routing, planning, workflow, atomic) plus the shared runtime protocol, and the right part shows the offline and online evolution loops that continuously refine the skills.

References. References encode the methodology behind a skill. They may include business-analysis principles, diagnostic procedures, statistical assumptions, visualization conventions, report structures, or domain-independent analysis patterns. Their purpose is to constrain the model toward expert analytical behavior without binding the method to a specific business table, metric, or organization.

Scripts. Scripts provide reusable operational assets for a skill, such as data-processing scripts, visualization helpers, prompt templates, tool-use conventions, and report templates. They make the method operational, but they are not run by Skill-Hub itself. Host is responsible for executing scripts and tool calls, while Skill-Hub only provides the assets and conventions that Host follows.

Together, these assets separate methodology from both evidence and runtime. Changing a metric definition should update DataBridge, not Skill-Hub; changing how a task is executed should update Host; changing the analytical procedure itself should update Skill-Hub.

4.3 Hierarchical Skill Organization

Skill-Hub organizes method assets into a hierarchy so that complex analytical workflows can be composed from reusable parts. The hierarchy moves from coarse-grained task understanding to fine-grained analytical operations, while keeping every level as a method description that Host can interpret.

Shared runtime protocol. The shared runtime protocol is the host-facing convention used by all skills. It is not an execution engine. Instead, it specifies how skill assets should expose inputs, outputs, intermediate artifacts, failure states, plan revisions, and pre-delivery quality checks. This protocol allows Host to interpret different skills consistently and prevents each skill from inventing its own artifact and state-passing format.

L0: Routing. Routing skills describe how to classify a user request into a task family and select a suitable downstream skill path. Common task families include metadata query, data query, BI business analysis, data exploration, statistical modeling, quantitative calculation, and report generation. The output of routing is a task-family decision and candidate method path, which Host can use when constructing the executable workflow.

L1: Planning. Planning skills describe how a task should be decomposed into an inspectable analytical plan. For BI business analysis, this may include identifying the business scenario, refining required metrics, choosing analysis perspectives, and specifying which evidence should be requested from DataBridge. The output is a plan specification rather than an executed plan; Host materializes it as a DAG and manages user inspection, revision, and execution.

L2: Workflow. Workflow skills describe complete analytical workflows by composing multiple atomic skills into a task-level procedure. They specify how analytical operations should be connected, what dependencies exist between them, what artifacts should be passed across steps, and what quality

checks are required before the workflow can produce a final answer. Representative workflows include BI-oriented workflows such as metric, cohort, conversion, and retention analysis, as well as broader workflows for data modeling, data-science analysis, and data fetching. These skills define the procedure for a task family; Host materializes the procedure into executable DAG actions.

L3: Atomic. Atomic skills describe the smallest reusable analytical operations. They cover recurring operation families such as metric calculation, thresholding and anomaly detection, dimensional drill-down and segmentation, attribution and causal analysis, distribution and funnel analysis, visualization, dashboard generation, and report generation. Atomic skills are designed to be reusable analytical building blocks, so different workflow skills can combine them with different evidence supplied by DataBridge.

This hierarchy gives Skill-Hub both structure and flexibility. High-level skills keep the overall analytical path consistent, while atomic skills make common operations reusable across workflow skills. Because business facts remain in DataBridge and execution remains in Host, the same method hierarchy can be reused across different data domains.

4.4 Skill Validation and Governance

Skill-Hub provides methodological control for open-form data analysis. Since most analytical tasks cannot be verified by a deterministic test, the system does not require each skill to manually embed a fixed checklist. Instead, QwenPaw-Data uses an automated validation scheme that derives task-specific checklists from the skill specification, references, expected artifacts, and current task context. These generated checklists make the analytical process inspectable without turning the skill itself into a static list of checks.

Validation does not prove that a business conclusion is absolutely correct. Instead, it verifies that the method was followed in a reproducible and reviewable way. During execution, a reflection subagent can use the generated checklist to examine whether the required evidence has been retrieved, whether key assumptions have been stated, whether intermediate artifacts have been produced, whether selected dimensions match the task intent, and whether the final report links conclusions back to supporting artifacts. This gives Skill-Hub a soft verification loop that partially compensates for the lack of compiler-like feedback in data analysis.

Governance controls how skills enter and change the repository. New or revised skills remain candidate assets until they pass review, evaluation, and versioning checks. Each active skill version should record what changed, why it changed, which evidence or feedback triggered the revision, how it was evaluated, and how it can be rolled back. This prevents temporary feedback, local preferences, or unsafe rollout results from silently changing global analytical behavior.

4.5 Skill Evolution

Skill-Hub evolves continuously so analytical methods remain effective in real usage. Together with the Learn stage of DataBridge, skill evolution forms the method half of the data-asset fly-wheel. DataBridge learns semantic evidence, traces, and rules; Skill-Hub learns reusable analytical methodology, workflow structures, references, and scripts.

Offline evolution. Offline evolution improves skills through benchmark tasks, historical cases, and rollout traces. Stable patterns across successful cases can reveal reusable workflows, better decomposition strategies, or common quality checks. Divergent or failed traces can reveal ambiguous decisions, missing constraints, or failure-prone method choices. These signals become candidate skill revisions rather than immediate changes to active skills.

Online evolution. Online evolution improves skills through human-in-the-loop feedback during real tasks. Users may correct an analysis procedure, request an additional perspective, reject an assumption, or ask for a different report structure. Such feedback is converted into candidate method updates only when it reflects a reusable analytical improvement rather than a one-off business fact. Business definitions and semantic corrections should be written to DataBridge, while reusable procedure changes belong to Skill-Hub.

Versioned promotion. Candidate revisions become active only after validation and versioning. A revision may add a new validation signal, refine a planning template, update a reference, improve a script, or introduce a new workflow. Once promoted, it becomes a new skill version that Host can use in future tasks. This process keeps the method library evolvable without sacrificing stability or auditability.

4.6 What Skill-Hub Guarantees

Through the interaction of method assets, hierarchy, validation, and evolution, Skill-Hub provides three guarantees for QwenPaw-Data:

- *Reusability*: analytical methods are separated from changing business facts, so the same skill can be reused across domains as long as DataBridge supplies the required evidence.
- *Methodological consistency*: similar tasks follow stable analytical procedures instead of relying on one-off model reasoning.
- *Verifiable evolvability*: skills can be improved through feedback and evaluation, but only through candidate revisions, validation, versioning, and governance.

Together, these guarantees allow Host to execute analytical workflows that are not only grounded by DataBridge, but also guided by reusable and continuously improving analytical methodology.

5 Host Runtime

5.1 Role: The Runtime Execution Subsystem

Host is the runtime execution subsystem of QwenPaw-Data. It takes governed evidence from DataBridge and method assets from Skill-Hub, and materializes them into executable analytical runs. In this subsystem, workflows become concrete runtime actions: task materialization, subagent dispatch, tool invocation, sandboxed execution, artifact recording, reflection, and failure recovery.

This separation keeps responsibilities explicit. DataBridge supplies governed semantic evidence, Skill-Hub supplies reusable method assets, and Host is the only execution subject: it coordinates subagents, runs tools and scripts in controlled environments, maintains runtime state, and records artifacts and execution traces. By making execution explicit as state, actions, artifacts, and events, Host realizes the *Controllable Long-horizon Execution Environment* highlight described in Section 1.4.

Figure 7 summarizes Host as the execution subsystem of QwenPaw-Data. The left side shows the specialized subagents and execution substrate managed by Host, while the right side shows the runtime lifecycle that turns method and evidence assets into executable, inspectable, and recoverable analytical workflows.

5.2 What Host Manages

Before introducing the runtime lifecycle, it is useful to clarify what kinds of runtime objects Host manages. Host does not manage business semantics or analytical methodology directly. Instead, it manages the execution substrate that allows those assets to be used safely in real tasks.

Execution graph and runtime state. Host represents a task as a directed acyclic graph (DAG), where nodes are executable actions and edges are dependencies between actions. The graph records which actions are ready, running, completed, failed, skipped, or waiting for user revision. Runtime state also includes session context, action inputs and outputs, dependency status, retry state, and the current position of the analytical workflow. This makes the progress of a long-running task inspectable rather than hidden inside a conversation.

Specialized subagents. Host organizes complex analysis through multiple specialized subagents rather than a single monolithic agent. A data-fetching subagent focuses on evidence-aware data access and retrieval. An analysis subagent focuses on metric computation, drill-down, attribution, comparison, and intermediate conclusion construction. A report subagent turns verified findings, charts, and artifacts into a decision-ready report. A reflection subagent uses automatically generated

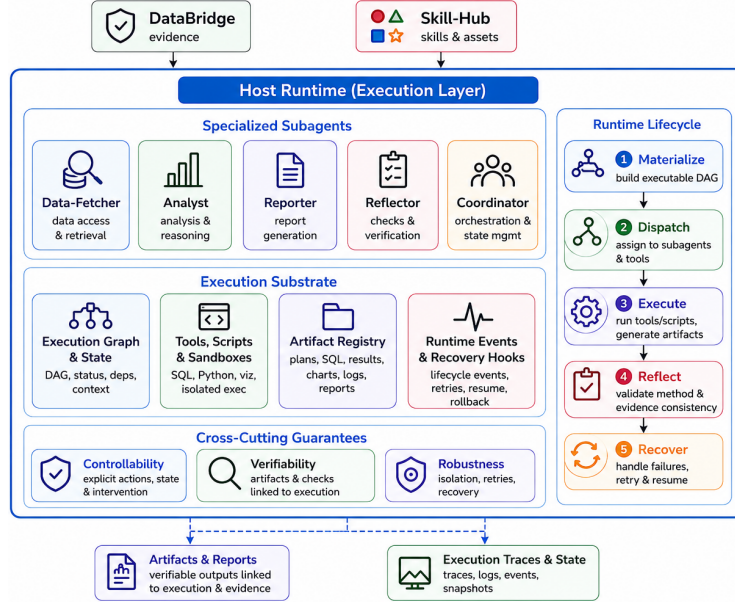


Figure 7: The overall structure of Host Runtime. DataBridge supplies governed evidence and Skill-Hub supplies skills and assets; Host materializes them into specialized subagent execution, execution substrate management, runtime lifecycle control, and cross-cutting guarantees.

checklists, derived from Skill-Hub assets and task context, to inspect whether the execution process followed the expected method and whether the final answer is supported by artifacts.

Tools, scripts, and sandboxes. Host is responsible for invoking tools and running scripts supplied or referenced by Skill-Hub. These executions may include SQL queries, Python data processing, visualization generation, file operations, and report rendering. To keep execution safe, Host runs tool-side operations inside isolated sandbox environments with separate workspaces, runtime environments, and permission boundaries. This allows analytical tasks to use flexible computational tools without exposing the host runtime or other sessions to unsafe side effects.

Artifact registry. Host maintains an artifact registry for the intermediate and final products generated during execution. Artifacts may include plans, SQL statements, query results, transformed datasets, charts, execution logs, generated checklists, reflection results, and final reports. Each artifact is linked back to the corresponding DAG node, tool invocation, and supporting evidence where possible. As a result, a conclusion can be traced to the concrete operations and artifacts that produced it.

Runtime events and recovery hooks. Host exposes runtime events and lifecycle hooks around key execution stages, such as task initialization, plan materialization, tool invocation, artifact generation, streaming, reflection, failure handling, and task completion. These hooks allow Host to observe abnormal conditions, preserve partial progress, retry failed actions, resume from recoverable checkpoints, and surface intervention points to users.

5.3 Runtime Execution Lifecycle

Host manages an analytical task through a runtime lifecycle that turns method and evidence assets into a controlled execution process. The lifecycle has five compact stages:

Materialize. Host converts the user request, selected Skill-Hub assets, and retrieved DataBridge evidence into an executable DAG. It interprets the workflow specification, identifies required inputs and intermediate artifacts, expands procedure steps into concrete actions, and attaches dependencies between actions.

Dispatch. Host assigns executable actions to appropriate subagents and tools according to DAG dependencies and action types. Data-fetching, analytical reasoning, report-writing, and reflection actions are routed to the corresponding subagents, while independent branches can run in parallel once their dependencies are satisfied.

Execute. Host carries out scheduled actions through tool calls, script execution, sandboxed computation, and artifact generation. SQL statements, query results, transformed datasets, charts, logs, and draft reports are written into the artifact registry and linked back to their DAG nodes.

Reflect. Host invokes the reflection subagent with task-specific checklists generated from Skill-Hub specifications, references, expected artifacts, and task context. The reflection process checks whether required evidence, assumptions, intermediate artifacts, selected dimensions, and final conclusions are consistent with the intended method.

Recover. Host handles failures, interruptions, and user revisions through runtime hooks and persisted state. Failed actions can be retried, branched, rolled back, or resumed from checkpoints without invalidating completed upstream artifacts.

5.4 What Host Guarantees

Through the interaction of execution graphs, specialized subagents, sandboxed tools, artifact registry, reflection, and recovery hooks, Host provides three guarantees for QwenPaw-Data:

- *Controllability:* analytical execution is represented as explicit actions, dependencies, state, and intervention points rather than as an implicit model conversation.
- *Verifiability:* intermediate artifacts, tool calls, generated checklists, reflection results, and final reports are linked to the execution process, so conclusions can be reviewed against their supporting evidence and artifacts.
- *Robustness:* sandbox isolation, lifecycle hooks, persisted state, retries, and checkpoint-based recovery prevent local tool or stream failures from collapsing the whole analytical workflow.

Together, these guarantees allow QwenPaw-Data to execute open-form data-analysis tasks as durable, inspectable, and recoverable workflows grounded by DataBridge and guided by Skill-Hub.

6 Evaluation

Having presented the design of QwenPaw-Data, we now examine whether it delivers measurable benefits in practice. Our evaluation is centered on two questions:

- *Q1. To what extent does QwenPaw-Data improve over a general-purpose agent in real enterprise analysis?*
- *Q2. How does each of its components contribute to these gains?*

To investigate these questions under realistic conditions rather than in a sandbox, we deployed QwenPaw-Data within an internal business line, where it serves business-intelligence (BI) users in their day-to-day analytical work and supplies the workloads used throughout this evaluation. There, analysts issue genuine business requests over live enterprise data hosted in a Hologres, whose heterogeneity, ambiguity, and continual evolution reflect the difficulties discussed in Section 1.3.

6.1 Enterprise Gains over a General Agent

To answer the first question, we evaluate QwenPaw-Data on tasks drawn from the BI users' daily usage, which fall into two categories. The first consists of 29 objective queries, such as NL2SQL and metric computation, each with an exact ground-truth answer. The second consists of 37 distinct open-ended analytical requests, spanning business metric monitoring and anomaly diagnosis, trend and growth analysis, periodic business reporting, and ad-hoc deep-dive attribution; each is a long-horizon task that requires from roughly 15 minutes to one hour of autonomous execution and culminates in a report.

Table 3: Component ablation on production BI tasks. All scores are percentages; higher is better.

Configuration	Analysis Breadth	Analysis Depth	Report Quality	Artifact Completeness
General-purpose agent	27.35	25.21	35.64	36.94
+ Skill-Hub	66.32	48.46	47.03	86.04
+ DataBridge	36.59	29.39	37.84	32.43
QwenPaw-Data (both)	79.63	62.60	58.09	86.04

Objective queries. We judge each objective query by exact agreement with a reference answer that has been reviewed by more than two domain experts. On these queries, QwenPaw-Data returns the correct result on about 96.5% of the cases. The main reason is that DataBridge grounds each query in the right metrics, dimensions, and tables, so the agent binds to the correct entities instead of guessing, while Skill-Hub contributes two assets: a robust data-retrieval pipeline that chains query rewriting, context gathering, disambiguation, SQL generation, and error recovery; and standardized metric-computation scripts. Together they let the agent follow a verified procedure rather than ad-hoc logic.

Open-ended queries. For each open-ended request, we let business users give a satisfaction score on a 100-point scale after they have used the result. Under this measure, the deployed QwenPaw-Data attains an average satisfaction of 71.6. When we replace it with a leading general-purpose agent from the strongest tier of current systems, one that lacks DataBridge and Skill-Hub, satisfaction drops to 34.1. To keep the comparison fair, both systems are driven by the same underlying LLM with fixed request parameters, so the reported gain is a conservative lower bound attributable purely to the harness. The gap traces directly to QwenPaw-Data’s three components. DataBridge grounds each request in the right metrics and dimensions, Skill-Hub supplies the expert analytical procedure, and the Host runtime keeps the long-horizon, multi-step execution reliable and every artifact traceable, so QwenPaw-Data tends to deliver a complete analytical loop: it presents the overall trend, abnormal fluctuations, and drill-down attribution layer by layer, and keeps every visualization consistent with the stated conclusion. The general-purpose agent, without grounded semantics or a codified procedure, guesses at the semantics and skips the deeper analytical steps, so its answers drift and stay shallow.

As an additional benefit, QwenPaw-Data brings two further gains. First, it annotates data lineage and query provenance throughout, so every result is fully auditable, which is essential in high-stakes data analysis where every conclusion must be traced back to its source before it can drive a business decision. Second, rather than leaving the LLM to explore the raw schema on its own, which is extremely costly when an industrial warehouse holds vast numbers of tables and columns, DataBridge supplies exactly the metrics, dimensions, and table context each step needs. In our measurements, this reduces context token consumption by roughly 42% on average.

6.2 Component Ablation

Having seen the end-to-end gains, we now answer the second question by isolating how each added component contributes. Holding the same underlying LLM fixed, we start from the general-purpose agent and enable Skill-Hub and DataBridge, individually and then together as the full QwenPaw-Data, on the same open-ended requests. To evaluate these configurations at scale under a single consistent standard, we ask the BI users to express their expectations for each request as explicit checkpoints along four dimensions, and an LLM-as-a-judge then scores every configuration against these checkpoints. The dimensions capture what business users care about: *analysis breadth*, whether the analysis covers the relevant metrics, dimensions, and segments rather than a single angle; *analysis depth*, whether it moves beyond surface numbers to anomaly attribution, root-cause decomposition, and actionable conclusions; *report quality*, whether the derived report is clear, well organized, and has every conclusion supported by data and visualizations; and *artifact completeness*, whether the expected artifacts, such as the analysis plan, query results, intermediate computation results, Python code, and charts, are all persisted to disk and thus traceable. Table 3 reports the results.

The general-purpose agent alone stays low across all four dimensions, consistent with the low satisfaction it received in the deployment comparison of Section 6.1. Adding Skill-Hub sharply lifts

Table 4: Results on public data-agent benchmarks. Higher is better; each column follows the benchmark’s official metric. The Human and current SOTA figures are taken from the respective benchmark papers [Lai et al., 2025, Lei et al., 2025].

Method	KramaBench	DAComp
Human	76.75	–
Current SOTA	55.83	50.84
QwenPaw-Data	68.32	62.38

analysis breadth and artifact completeness: it supplies carefully-curated analytical skills that walk the agent through the complete analysis flow layer by layer, while its runtime skills enforce persistence conventions that write every intermediate artifact to disk and keep it traceable. Yet analysis depth and report quality stay limited, because without grounded metric and dimension semantics and the relationships among them, the agent is unsure of the right direction and its analysis can drift, for example failing to drill down into the secondary or even tertiary dimensions, which in turn leaves the report thin. Adding DataBridge alone grounds the semantics but yields only modest gains, because without a codified methodology the agent skips the complete analytical steps. Only with both does QwenPaw-Data reach the top level across all four dimensions, and the combined gain clearly exceeds the sum of the two components applied alone, showing that grounding evidence and methodology are not merely complementary but mutually reinforcing, delivering their full value only when present together.

6.3 Reproducibility on Open-Source Benchmarks

The evaluations in Sections 6.1 and 6.2 run on proprietary enterprise workloads that cannot be publicly disclosed. To let others verify QwenPaw-Data’s underlying capability in a reproducible and comparable setting, we also report results on two public data-agent benchmarks, KramaBench [Lai et al., 2025] and DAComp [Lei et al., 2025], which echo different facets of enterprise analysis. KramaBench builds end-to-end data analysis pipelines over heterogeneous, multi-source data lakes, which requires sifting the useful evidence out of a large volume of raw data, and thus mirrors the data part of our real tasks. DAComp instead mimics the data-intelligence lifecycle over larger databases and calls for more open-ended reasoning, closer to the intent and scale of real analytical requests. Both go beyond a single lookup and require multi-step pipelines from raw data to insight.

As shown in Table 4, QwenPaw-Data leads the current state of the art on both benchmarks, and on KramaBench it closes much of the remaining gap to human experts. It reaches this with the same design that drives its enterprise gains: through a standardized data-retrieval and discovery pipeline it narrows a sprawling data lake to the most useful information, and it keeps the multi-step analysis on a disciplined path from raw data to insight. That this lead holds on public data confirms that the underlying capability generalizes beyond our private workloads.

Summary. Overall, these results answer the two questions posed at the outset. In real enterprise analysis, QwenPaw-Data resolves objective queries with high accuracy and substantially outperforms a strong general-purpose agent on open-ended tasks (Q1); this advantage stems from the interplay between grounded enterprise semantics and codified analytical skills, neither of which suffices in isolation (Q2); and its lead on public benchmarks shows that the capability is reproducible and generalizes beyond our own deployment.

7 Future Work

The current version of QwenPaw-Data establishes the core architecture for agentic data analysis: DataBridge grounds execution with semantic evidence, Skill-Hub standardizes reusable analytical methods, and Host turns both into controllable runtime workflows. Future work will follow the same architectural line, moving from a reliable analysis framework toward a continuously improving enterprise data intelligence system.

Strengthening the semantic–method–runtime flywheel. The first direction is to make the three core components improve each other more effectively. DataBridge will continue to support richer semantic mining, automatic graph refinement, and semantic evolution from execution traces and user feedback. Skill-Hub will focus on skill transfer and cold-start for new domains, so existing analytical methods can be adapted rather than rebuilt from scratch. Host will improve execution efficiency, deployment flexibility, and interoperability with emerging agent runtimes. Together, these improvements make the framework more reusable across domains and more adaptive to real enterprise usage.

Improving interactive and self-verifying execution. The second direction is to make long-running analysis more inspectable and correctable while it is being executed. Future versions of QwenPaw-Data will support stronger human-in-the-loop intervention, allowing users to inspect intermediate artifacts, revise execution plans, correct analytical procedures, and guide subsequent actions without restarting the workflow. We will also improve self-judging mechanisms that estimate result confidence by combining semantic consistency, execution evidence, reflection outcomes, and historical feedback. This turns verification from a final manual review into a runtime capability.

Scaling toward enterprise data intelligence. The third direction is to extend QwenPaw-Data from an analytical framework toward a data brain for enterprise decision-making and data management. This direction focuses on making insight discovery, action recommendation, workflow coordination, and data-asset optimization more systematic, governed, and enterprise-scale. To support this direction, we will further strengthen tenant isolation, fine-grained permission control, secure execution environments, and enterprise-grade governance for large-scale multi-tenant deployments.

8 Conclusion

In this report, we presented QwenPaw-Data, an end-to-end framework for agentic data analysis organized around three AI-native architectural questions: *what facts to use*, *how to analyze*, and *how to run the resulting workflow*. DataBridge provides governed semantic evidence, Skill-Hub provides reusable analytical methodology, and Host provides the runtime execution subsystem that turns evidence and methods into inspectable workflows. Together, the three components address the core difficulty of enterprise data analysis: the system must ground business concepts, follow consistent analytical procedures, and execute long-running workflows in a controllable environment.

The resulting architecture gives QwenPaw-Data three complementary forms of reliability. DataBridge improves the groundedness and freshness of evidence; Skill-Hub improves the reusability and consistency of analytical methods; and Host improves the controllability, verifiability, and robustness of execution. This modular design provides a practical foundation for continuously improving data-analysis agents and for scaling them toward governed, enterprise-level data intelligence.

References

- Alibaba Cloud. Quick BI Intelligent Q: A New Data Consumption Paradigm. <https://help.aliyun.com/en/quick-bi/product-overview/intelligentq>, 2026. Alibaba Intelligent Q documentation, accessed July 2026.
- Alibaba Group. Qwenpaw. <https://github.com/agentscope-ai/QwenPaw>, 2026. GitHub repository, accessed July 2026.
- Amazon. Generative BI with QuickSight. <https://docs.aws.amazon.com/quick/latest/userguide/quicksight-gen-bi.html>, 2026. Amazon QuickSight documentation, accessed July 2026.
- Anthropic. How anthropic enables self-service data analytics with claude. <https://claude.com/blog/how-anthropic-enables-self-service-data-analytics-with-claude>, 2026. Anthropic blog, accessed July 2026.
- Databricks. Genie Agents Concepts. <https://docs.databricks.com/aws/en/genie/concepts>, 2026a. Databricks Genie Agents documentation, accessed July 2026.

- Databricks. Genie Ontology. <https://docs.databricks.com/aws/en/genie-one/chat#genie-ontology>, 2026b. Databricks Genie Ontology documentation, accessed July 2026.
- Google. Conversational Analytics Overview. <https://docs.cloud.google.com/bigquery/docs/conversational-analytics>, 2026. BigQuery Conversational Analytics documentation, accessed July 2026.
- IBM. IBM Cognos Analytics AI Assistant. <https://www.ibm.com/docs/en/cognos-analytics/12.0.x?topic=panel-getting-started-assistant>, 2026. IBM Cognos Analytics documentation, accessed July 2026.
- Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Sivaprasad Sudhir, Om Chabra, Anna Zeng, Anton A. Zabreyko, Chenning Li, Ferdi Kossmann, Jialin Ding, Jun Chen, Markos Markakis, Matthew Russo, Weiyang Wang, Ziniu Wu, Michael J. Cafarella, Lei Cao, Samuel Madden, and Tim Kraska. Kramabench: A benchmark for AI systems on data-to-insight pipelines over data lakes. *CoRR*, abs/2506.06541, 2025.
- LangChain. State of agent engineering. <https://www.langchain.com/state-of-agent-engineering>, 2026. LangChain blog, accessed July 2026.
- Fangyu Lei, Jinxiang Meng, Yiming Huang, Junjie Zhao, Yitong Zhang, Jianwen Luo, Xin Zou, Ruiyi Yang, Wenbo Shi, Yan Gao, Shizhu He, Zuo Wang, Qian Liu, Yang Wang, Ke Wang, Jun Zhao, and Kang Liu. Dacomp: Benchmarking data agents across the full data intelligence lifecycle. *CoRR*, abs/2512.04324, 2025.
- Ruiying Ma, Shreya Shankar, Ruiqi Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G. Parameswaran. Can AI agents answer your data questions? A benchmark for data agents. *CoRR*, abs/2603.20576, 2026.
- Microsoft. Fabric Data Agent Creation. <https://learn.microsoft.com/en-us/fabric/data-science/concept-data-agent>, 2026a. Microsoft Fabric Data Agent documentation, accessed July 2026.
- Microsoft. What Is Ontology (Preview)? <https://learn.microsoft.com/en-us/fabric/iq/ontology/overview>, 2026b. Microsoft Fabric IQ Ontology documentation, accessed July 2026.
- OpenClaw Contributors. Openclaw. <https://github.com/openclaw/openclaw>, 2026. GitHub repository, accessed July 2026.
- Oracle. About Oracle Analytics AI Agents. <https://docs.oracle.com/en/cloud/paas/analytics-cloud/acubi/oracle-analytics-ai-agents.html>, 2026. Oracle Analytics AI Agents documentation, accessed July 2026.
- Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: decomposed in-context learning of text-to-sql with self-correction. In *NeurIPS*, 2023.
- Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan Ö. Arik. CHASE-SQL: multi-path reasoning and preference optimized candidate selection in text-to-sql. In *ICLR*. OpenReview.net, 2025.
- Snowflake. Cortex Analyst. <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst>, 2026. Snowflake Cortex Analyst documentation, accessed July 2026.
- Zhaoyan Sun, Shan Zhong, Daizhou Wen, Jiaying Han, Guoliang Li, Ying Yan, Peng Zhang, Yu Su, Xiang Qi, Baolin Sun, Chengyuan Yang, Tao Fang, and Huaiyu Ruan. Agenticdatabench: A comprehensive benchmark for data agents. *CoRR*, abs/2607.01647, 2026.
- Volcengine. Intelligent Analytics Agent Overview. <https://www.volcengine.com/docs/86760/1874950>, 2026. Volcengine Intelligent Analytics Agent documentation, accessed July 2026.